secure computing - seccomp

Mateusz Karłowski

2005

CPUShare

- Founded by Andrea Arcangeli in 2005
- Marketplace for idle CPU cycles — owners rent out their machines
- Clients submit jobs and pay for compute time
- Goal: make unused hardware profitable for Linux users

The Problem

- Client code runs directly on the owner's machine
- Nothing stops a malicious client from calling **any** syscall
- `fork` a hundred processes, `open` private files, `connect` to the network…
- Containers and KVM don't exist yet — virtualization is too slow
- How do you run **untrusted code** safely with zero overhead?

The Solution

For pure computation, a process only ever needs:

Syscall	Why
`read`	receive input data
`write`	return results
`exit`	terminate cleanly
`sigreturn`	return from signal handler

> Lock the process to these 4 syscalls. Anything else → **instant kill**.

seccomp strict: first iteration

Block all syscalls except `read`, `write`, `exit`, and `sigreturn`. This is the **seccomp strict** mode, introduced in 2005.

```
echo 1 > /proc/<PID>/seccomp
```

2007

Interface change: prctl

The `/proc/<PID>/seccomp` interface was removed — replaced with `prctl`:

```
#include <sys/prctl.h>
#include <linux/seccomp.h>

prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT);
```

> Key difference: a process now opts **itself** in — no external actor needed

Demo

1992

tcpdump

- Network debugging tool — captures packets on a live interface
- Developed at Lawrence Berkeley National Laboratory
- Lets you filter traffic: “show me only HTTP”, “only packets from 10.0.0.1”
- Became the standard tool for network analysis and troubleshooting

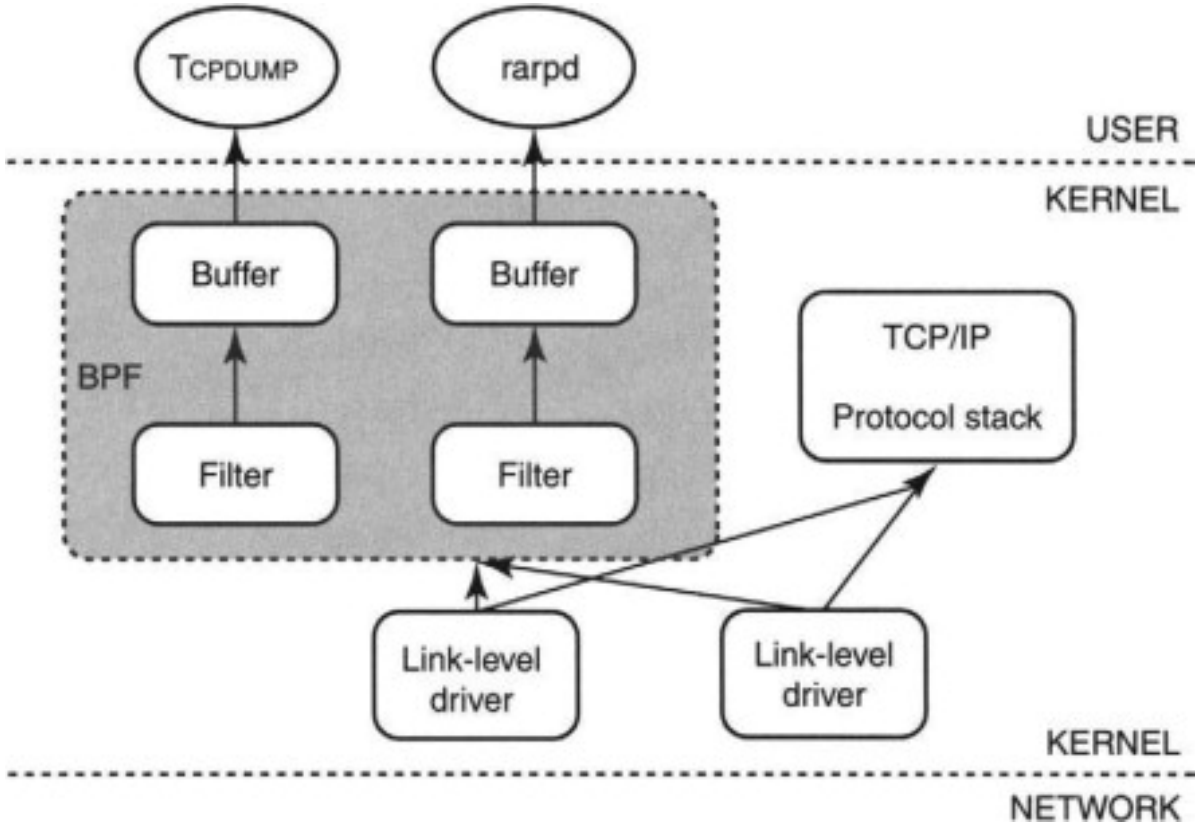
The Problem

- Kernel copies **every single packet** to user space
- tcpdump then checks if the packet matches the filter — and discards most of them
- On a busy network: millions of useless copies per second
- High CPU usage, dropped packets, poor performance

> Filtering happens **too late** — after the expensive copy to user space

The Solution: BPF

- McCanne & Jacobson (1992 USENIX paper): move the filter **into the kernel**
- User supplies a small filter program — kernel runs it on each packet **before** copying
- Only matching packets ever reach user space
- Huge performance gain — became the foundation of tcpdump and libpcap
- Merged into Linux in 1997 (kernel 2.1.75)



Snort Intrusion Detection 2.0

BPF assembly language

- Tiny virtual machine running **inside the kernel**
- Registers: `A` (accumulator), `X` (index)
- Small scratch memory: `M[0..15]`
- No loops — program always terminates
- Kernel verifies the program before running it

BPF filter: UDP port 53

```
ldh  [12]                ; load EtherType (2 bytes at offset 12)
jeq  #0x0800, +0, drop   ; must be IPv4
ldb  [23]                ; load IP protocol (1 byte at offset 23)
jeq  #17,    +0, drop    ; must be UDP (protocol = 17)
ldh  [36]                ; load UDP dest port (2 bytes at offset 36)
jeq  #53,    +0, drop    ; must be port 53
ret  #0xFFFFFFFF         ; accept — return entire packet
drop: ret  #0            ; discard
```

Demo

BPF filter in C: UDP port 53

```
/* BPF_STMT(opcode, operand)       — unconditional instruction   */
/* BPF_JUMP(opcode, val, jt, jf)  — jt: jump if true, jf: false */

struct sock_filter filter[] = {
    BPF_STMT(BPF_LD  | BPF_H   | BPF_ABS, 12),         /* ldh [12]  EtherType    */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, 0x0800, 0, 4), /* jeq IPv4, else drop    */
    BPF_STMT(BPF_LD  | BPF_B   | BPF_ABS, 23),         /* ldb [23]  IP protocol  */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, 17,    0, 2),  /* jeq UDP,  else drop    */
    BPF_STMT(BPF_LD  | BPF_H   | BPF_ABS, 36),         /* ldh [36]  UDP dst port */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, 53,    0, 1),  /* jeq 53,   else drop    */
    BPF_STMT(BPF_RET | BPF_K, 0xFFFFFFFF),              /* accept                 */
    BPF_STMT(BPF_RET | BPF_K, 0),                       /* drop                   */
};
```

2012

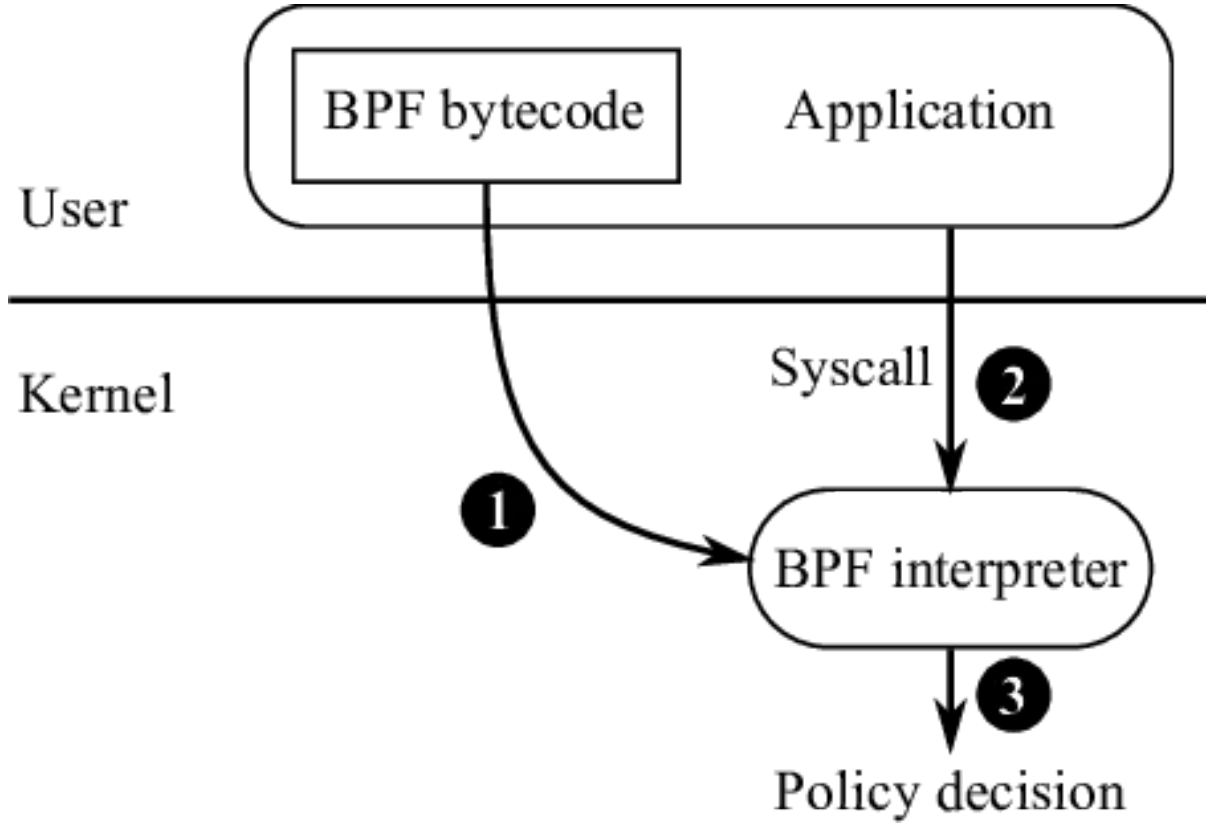
seccomp-bpf

In 2012, seccomp was extended to allow **custom BPF filters** — not just the strict mode.

> This allows for much more flexible policies — e.g. “allow `open` only if the filename is `/tmp/data.txt`”

What can you do with a syscall?

- `SECCOMP_RET_ALLOW` - pass through — syscall executes normally
- `SECCOMP_RET_KILL` - process killed immediately with SIGSYS
- `SECCOMP_RET_ERRNO` - syscall returns −1, errno set by filter
- `SECCOMP_RET_TRAP` - deliver SIGSYS to process
- `SECCOMP_RET_TRACE` - notify attached ptrace tracer
- `SECCOMP_RET_LOG` - allow and log to audit log



Zachary Tatlock - Jitk: A Trustworthy In-Kernel Interpreter Infrastructure

Filter inheritance

- seccomp filter is **inherited** by child processes across `fork` and `exec`
- Child can only **add** filters on top — never remove or relax them
- Filters stack: all filters in the chain are evaluated, most restrictive wins
- A sandboxed process cannot escape by spawning a child

seccomp-bpf filter: block socket

BPF input is `struct seccomp_data` — not a packet:

```
struct sock_filter filter[] = {
    /* load syscall number */
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, nr)),
    /* if not socket → allow */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_socket, 0, 3),
    /* load first argument (domain) */
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, args[0])),
    /* if domain != AF_INET → allow */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, AF_INET, 0, 1),
    /* socket(AF_INET, ...) → notify user space */
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ERRNO(EPERM)),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
};
```

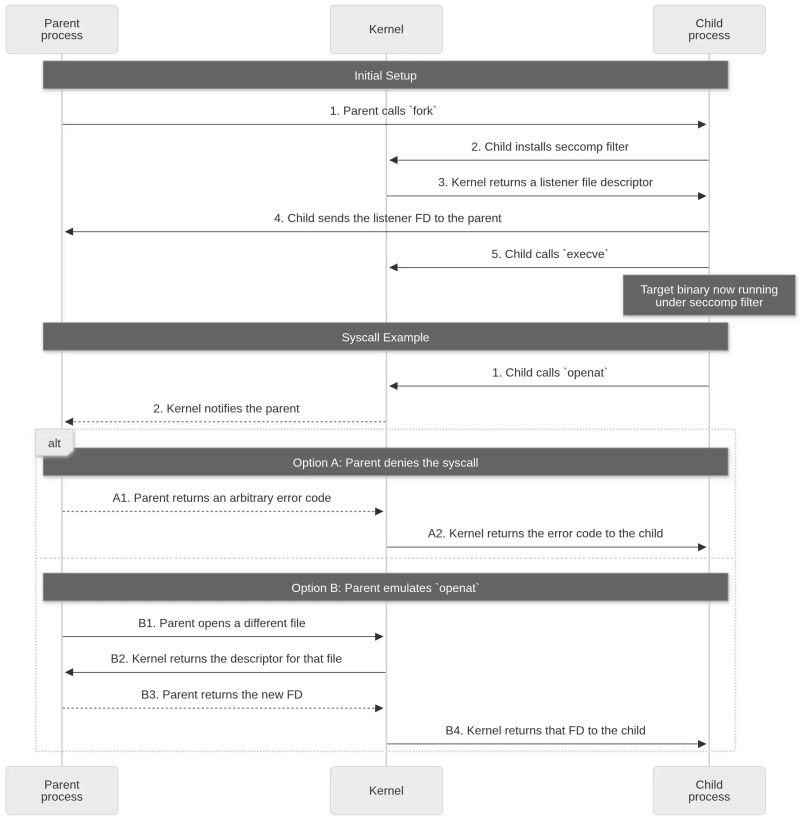
Demo

2019

seccomp notify

- `SECCOMP_RET_USER_NOTIF` — Linux 5.0 (2019)
- Instead of killing or blocking, **pause the process** and notify user space
- A supervisor process receives the syscall details and decides what to do
- Can **allow**, **deny**, or **emulate** the syscall — return any value back to the process

> First time seccomp can **change** syscall behavior — not just allow or kill



Kyle Avery - Linux Process Injection via Seccomp Notifier

seccomp notify: example (libseccomp-go)

```go
req, err := libseccomp.NotifReceive(notifyFd)
if err != nil {
  panic(err)
}
fmt.Printf("Received notification for syscall %d\n", req.Data.Syscall)
newFd, err := InjectFd(notifyFd, req, int(file.Fd()))
if err != nil {
  panic(err)
}
fmt.Printf("Injected fd %d into the process\n", newFd)
err = libseccomp.NotifRespond(notifyFd, &libseccomp.ScmpNotifResp{
  ID:     req.ID,
  Val:    uint64(newFd),
  Error:  0,
  Flags:  0,
})
if err != nil {
  panic(err)
}
```

Demo

seccomp in the action

Project	How
Chrome/Chromium	renderer sandbox since 2012 — one of the first large users
Docker	default JSON seccomp profile, blocks 40 dangerous syscalls
systemd	`SystemCallFilter=` in unit files via libseccomp
OpenSSH	sandbox for pre-auth privilege separation (since 9.0)
Firefox	content process sandbox on Linux
Flatpak/Bubblewrap	application sandboxing on the desktop
LXD / Podman	seccomp notify for syscall emulation in rootless containers

gvisor	kernel for containers - seccomp for syscall interception and emulation

Sources

- *LWN(2015), “A seccomp overview”*,
- *LWN(2005), “Securely renting out your CPU with Linux”*,
- *Linux kernel documentation, “BPF instruction set”*,
- *man seccomp_unotify(2)*,
- *man seccomp(2)*,
- *libseccomp documentation*,